

Controle de Versão

com Git & GitHub

Do primeiro **git init** ao **git push**: como rastrear cada alteração do seu projeto e colaborar sem perder trabalho.

Sumário

1. O que é controle de versão
2. Git × GitHub: qual a diferença
3. Criando a conta e configurando o Git
4. Repositório e o primeiro projeto
5. O fluxo de trabalho: as Três Árvores
6. Conectando o repositório ao GitHub

1. O que é controle de versão?

O **controle de versão** — conhecido pelas siglas **VCS** (Version Control System) ou **SCM** (Source Control Management) — é um sistema que rastreia o histórico de alterações em arquivos e coordena o trabalho de várias pessoas sobre eles. Na prática, ele registra cada modificação de um projeto ao longo de uma linha do tempo.

No desenvolvimento de software moderno, o controle de versão é considerado **essencial**. Veja os cinco motivos principais:

1.1 • Trabalho em equipe sem bloqueios

Sem um VCS, quando várias pessoas editam o mesmo arquivo, o trabalho pode ser **sobrescrito** (alguém perde o que fez) ou o arquivo fica **bloqueado** para edição, travando a produtividade. O controle de versão deixa todos trabalharem em paralelo e combina as alterações de forma organizada.

1.2 • Histórico completo e recuperação (backup)

O sistema registra **o que mudou, quando e por quem**. Se um bug aparece, a equipe volta atrás e recupera o estado exato do passado — e ainda descobre a origem da falha comparando versões.

1.3 • Segurança para experimentar com branches

As ramificações (**branches**) são linhas paralelas de desenvolvimento. Elas isolam um trabalho — nova funcionalidade, correção ou teste — sem risco de quebrar o código principal que já está em produção.

1.4 • Redundância contra falhas (distribuído)

Em sistemas **distribuídos** como o Git, cada pessoa tem uma **cópia completa do repositório com todo o histórico** na própria máquina. Não depende de rede constante e, se o servidor central falhar, o projeto é restaurado a partir de qualquer cópia local.

1.5 • Qualidade e agilidade

Com a complexidade crescente dos softwares, controle de versão, automação e colaboração transparente são o único caminho viável para **garantir qualidade, agilidade e eficiência** na entrega.



Resumo: cada ponto salvo no Git é como uma fotografia (um **commit**) do projeto num instante do tempo — e você pode voltar a qualquer um deles quando quiser.

2. Git × GitHub: qual a diferença?

Eles andam lado a lado, mas são **ferramentas distintas e complementares**. A forma mais simples de memorizar: **o Git é a ferramenta; o GitHub é a plataforma**.

Git — a ferramenta de controle de versão

- **O que é:** software de controle de versão distribuído, de código aberto, que roda localmente e opera por linha de comando.
- **Função:** rastrear o histórico e tirar "fotos" (commits) do estado dos arquivos.
- **Como funciona:** de forma distribuída — cada dev tem uma cópia exata do repositório, podendo trabalhar offline.
- **Papel:** atua como o cliente local.

GitHub — a plataforma de hospedagem

- **O que é:** serviço online (nuvem) que hospeda os repositórios gerenciados pelo Git.
- **Função:** centralizar o código na nuvem, compartilhar trabalho e construir portfólio.
- **Recursos:** pull requests, issues e automação com GitHub Actions.
- **Papel:** atua como o servidor remoto.

Aspecto	Git	GitHub
O que é	Controle de versão distribuído, código aberto	Serviço online que hospeda repositórios Git
Onde roda	Localmente, no seu computador	Na nuvem
Função	Rastrear histórico e tirar "fotos" (commits)	Centralizar, compartilhar e colaborar
Recursos	Branches, commits, log (terminal)	Pull requests, issues, Actions, portfólio
Papel	Cliente local	Servidor remoto



Resumo da relação: você pode usar o Git sozinho, offline, sem nunca abrir o GitHub. Mas para colaborar ou publicar, usa o **git push** e envia seus arquivos e histórico para um repositório hospedado no GitHub.

3. Criando a conta e configurando o Git

3.1 · Criando a conta no GitHub

1. **Acesse o site oficial:** entre em github.com.
2. **Inicie a inscrição:** clique em Sign up, no canto superior direito.
3. **Preencha os dados:** e-mail, uma senha segura e um nome de usuário (username).
4. **Verificação:** resolva o CAPTCHA quando solicitado.

5. **Escolha do plano:** selecione o Free — repositórios ilimitados para uso pessoal e colaborativo.

6. **Confirme o e-mail:** abra sua caixa de entrada e ative a conta.

3.2 · Configurando o Git no seu computador

Depois de criar a conta, identifique suas alterações locais para que o Git saiba quem fez cada commit. Abra o terminal (Linux/macOS) ou o **Git Bash** (Windows) e rode:

```
$ git config --global user.name "Seu Nome de Usuário"
$ git config --global user.email "seu-email@exemplo.com"
# use o MESMO e-mail cadastrado na sua conta do GitHub
```

3.3 · Autenticação para o git push

O GitHub **não aceita senhas tradicionais** na linha de comando. Configure uma das opções seguras abaixo:

- **Personal Access Token (PAT):** gerado em Settings › Developer Settings › Personal Access Tokens. Substitui a senha ao rodar git push.
- **Chave SSH:** cadastre uma chave SSH na conta para autenticar sem digitar credenciais toda vez.

✓ Com a **conta criada**, o **Git configurado** e a **chave/token definida**, sua máquina está pronta para git init, git add, git commit e git push.

4. Repositório e o primeiro projeto

Um **repositório** (o **repo**) é uma pasta especial que guarda todos os arquivos do projeto mais o histórico completo de alterações. Como o Git é distribuído, você tem uma cópia idêntica de tudo na sua máquina local.

Por trás dos panos, o repositório é monitorado por uma pasta oculta chamada **.git**, criada automaticamente na raiz. Ela é o "cérebro" do controle de versão: guarda commits, tags, ramificações e configurações.

Método 1 · Criar do zero (git init)

```
$ cd caminho/da/sua-pasta
$ git init
Initialized empty Git repository in .../.git/
# dica: git init nome-do-projeto cria a pasta e inicializa de uma vez
```

A partir daí, a pasta **.git** é criada e o Git passa a monitorar qualquer modificação de arquivos naquele diretório.

Método 2 · Clonar um projeto existente (git clone)

```
$ git clone https://github.com/usuario/projeto.git
Cloning into 'projeto'...
# baixa TODOS os arquivos + TODO o histórico de uma vez
```

5. O fluxo de trabalho: as Três Árvores

Para registrar versões, seu código passa por um ciclo entre as três "áreas" do Git — as **Três Árvores**:

- **Working Directory (Diretório de Trabalho)**: a pasta física onde você cria, edita e remove arquivos em tempo real.
- **Staging Area (Área de Preparação / Index)**: o local intermediário onde você prepara as alterações do próximo commit.
- **HEAD (Repositório / Commit)**: o repositório consolidado com seu último registro de versão oficial.

O ciclo prático, passo a passo

Passo 1 — Verifique o estado: veja o que mudou e ainda não foi salvo.

```
$ git status
```

Passo 2 — Adicione à preparação: diga ao Git o que entra no próximo commit.

```
$ git add README.md    # um arquivo específico
$ git add .             # todas as modificações da pasta
```

Passo 3 — Salve a versão (commit): uma "foto" do código com mensagem clara.

```
$ git commit -m "Cria o arquivo de documentação README"
```

Passo 4 — Acompanhe o progresso: veja a linha do tempo dos salvamentos.

```
$ git log
```

 **Fluxo resumido:** `git add` prepara as alterações; `git commit` as consolida no histórico local.

6. Conectando o repositório local ao GitHub

Para enviar seu repositório local à nuvem e deixá-lo acessível no GitHub, siga as etapas finais:

1. **Crie o repositório remoto:** no GitHub, clique em New, dê um nome e confirme.
2. **Copie a URL** do repositório remoto recém-criado.
3. **Vincule os dois repositórios** no terminal local.
4. **Defina a ramificação principal** como main.
5. **Envie o código** local para a nuvem.

```
$ git remote add origin https://github.com/seu-usuario/seu-projeto.git
$ git branch -M main
$ git push -u origin main
Enumerating objects... done.
```

Branch 'main' set up to track remote branch 'main' from 'origin'.

 **Pronto!** Seu repositório local está criado, ativo e sincronizado com o GitHub. A partir daqui, todo novo commit pode ser enviado com um simples **git push**.

Resumo · o caminho completo

git init → git add . → git commit → git push → nuvem 

Versione tudo. Colabore sem medo. Publique com um push.